Freezing Strategies in Transfer Learning for Skin Lesion Classification

Deep Learning Project Report S26

Harsh Naik
University of Bern
harsh.nayak@students.unibe.ch

Avin Pathak
University of Bern
avin.pathak@students.unibe.ch

Daksahaini Chellappah
University of Bern
daksahaini.chellappah@students.unibe.ch

Jasmin Injodikaran
University of Bern
jasmin.injodikaran@students.unibe.ch

1 Introduction and Background

Transfer learning has become a key method in deep learning, allowing models trained on large datasets to be efficiently adapted to new domains where data is often limited. A central question in this process is how much of the previously learned knowledge should be preserved, frozen, or fine-tuned when moving to a different task Yosinski et al. [2014].

This project examines different strategies for freezing layers in transfer learning applied to the classification of skin lesions, a task of clear clinical importance, where labeled data are scarce and accurate predictions can have direct diagnostic value Esteva et al. [2017]. By systematically testing various freezing configurations, we aim to better understand the balance between preserving general visual representations and adapting to task-specific features.

We take inspiration from an analogy in medical training: when a general practitioner specializes in dermatology, they keep their foundational understanding of medicine (such as anatomy and physiology) but develop new, focused expertise. In a similar way, a pre-trained neural network retains basic visual patterns like edges and textures that remain useful, while its higher-level layers often need to be fine-tuned to capture the subtle distinctions between different kinds of skin lesions.

> *"When adapting a pre-trained model to a new task, how much of its previously learned knowledge should be retained?"*

2 Method

We use the HAM10000 dataset (Human Against Machine with 10,000 training images), accessible via Kaggle and TensorFlow Datasets Tschandl et al. [2018]. Key dataset properties:

- 10,015 dermoscopic images across 7 classes: melanoma (MEL), melanocytic nevi (NV), basal cell carcinoma (BCC), actinic keratosis (AKIEC), benign keratosis (BKL), dermatofibroma (DF), and vascular lesions (VASC).
- Significant class imbalance (NV dominates with $\sim$67% of samples).
- Standard train/validation/test splits are applied with stratified sampling.
- Preprocessing: resize to 224×224, ImageNet normalization, augmentation (flip, rotation, color jitter).

Due to class imbalance, we apply a class-weighted loss during training and report balanced accuracy and macro-averaged AUC as primary metrics.

2.1 Model Architecture

We use EfficientNet-B0 pre-trained on ImageNet as our primary backbone, due to its favorable accuracy–efficiency trade-off and widespread use in medical imaging benchmarks. ResNet18 is used for cross-architecture validation in a subset of experiments, allowing us to separate strategy effects from architecture effects.

We compare three freezing strategies:

Table 1: Three transfer-learning strategies evaluated on each backbone.

Strategy	Description	Expected Behaviour
S1 – From Scratch	All weights randomly initialised; full network trained on HAM10000	Weakest performance; lower bound baseline due to limited data
S2 – Full Freeze	All pre-trained weights frozen; only the classification head is trained	Fast training; may underfit if ImageNet features are insufficient
S3 – Gradual Unfreeze	Start with frozen backbone; progressively unfreeze layer blocks every few epochs	Best generalisation; balances retained knowledge with task-specific adaptation

2.2 Training Setup

In this project, we train a neural network to classify images of skin lesions using the HAM10000 dataset. This dataset contains just over $10{,}000$ images divided into seven different classes (MEL, NV, BCC, AKIEC, BKL, DF, VASC). One important challenge is that the dataset is heavily imbalanced — melanocytic nevi (NV) alone account for roughly 67% of the samples. To handle this, we split the dataset into training, validation, and test sets (80%, 10%, and 10%) using stratified sampling, which keeps the same class distribution in each split. This ensures that the model is evaluated fairly across all classes.

Before training, the images are preprocessed. First, all images are resized to 224×224 pixels so they can be used as input to the neural network. Next, the images are normalised using the standard ImageNet mean and standard deviation, since both backbones (EfficientNet-B0 and ResNet18) are pre-trained on ImageNet. During training only, we apply data augmentation in the form of random horizontal flips and random rotations of up to $\pm 10^\circ$. This helps the model generalise better by exposing it to mild geometric variation in the data.

To train the model, we use the cross-entropy loss function, which measures how wrong the model's predictions are. Because the dataset is imbalanced, we use class weights computed on the training split with sklearn's `balanced` scheme so that mistakes on rare classes (DF, VASC, AKIEC) are penalised more than mistakes on common ones (NV). This helps the model learn all classes more equally.

We use the Adam optimiser to update the model during training. Adam is a commonly used optimisation method that automatically adjusts how fast each parameter learns. It works well for deep learning tasks and converges quickly. The initial learning rate is set to 10^{-3} while only the classification head is trainable. For the gradual-unfreeze strategy (S3), the optimiser is rebuilt at a smaller learning rate of 10^{-4} after each unfreezing event, so newly trainable feature blocks are updated more gently and the pre-trained features are not disturbed.

Finally, we define several hyperparameters, which are settings chosen before training. These include a batch size of 32 (how many images are processed at once), 5 training epochs per strategy, and an unfreeze interval of 1 epoch for S3 (one feature block is unfrozen at the start of every epoch after the first). Rather than using early stopping, we save the best checkpoint by validation accuracy after every epoch, so the final test evaluation always uses the strongest model observed during training.

2.3 Implementation Details

The code is organised as a small package with namespace imports (`data/`, `models/`, `evaluation/`); all entry points (`train_s1.py`, `train_s2.py`, `train_s3.py`, `main.py`) are thin wrappers around `models.train.run_strategy`, which loads data, builds the requested model, trains, evalu-

ates on the held-out test split, and persists training curves, confusion matrices, and a metrics JSON. Optimiser parameter groups are built with `filter(lambda p: p.requires_grad, model.parameters())` so frozen parameters never receive gradients; the optimiser is rebuilt after each S3 unfreezing event so new trainable parameters are picked up. The S3 unfreeze logic is backbone-specific: for EfficientNet, blocks of `model.features.children()` are unfrozen top-down; for ResNet18, `layer4` → `layer1` via `unfreeze_resnet_block`.

3 Results

Table 2 reports the test-set metrics for the three freezing strategies on the EfficientNet-B0 backbone. The ranking is monotone and unambiguous: S3 $>$ S2 $\gg$ S1.

Table 2: Test-set results on HAM10000 (EfficientNet-B0, 5 epochs). Higher is better for all metrics.

Strategy	Balanced Acc.	Macro F_1	Macro AUC
S1 – From Scratch	0.375	0.225	0.811
S2 – Full Freeze	0.639	0.472	**0.916**
S3 – Gradual Unfreeze	**0.661**	**0.495**	0.915

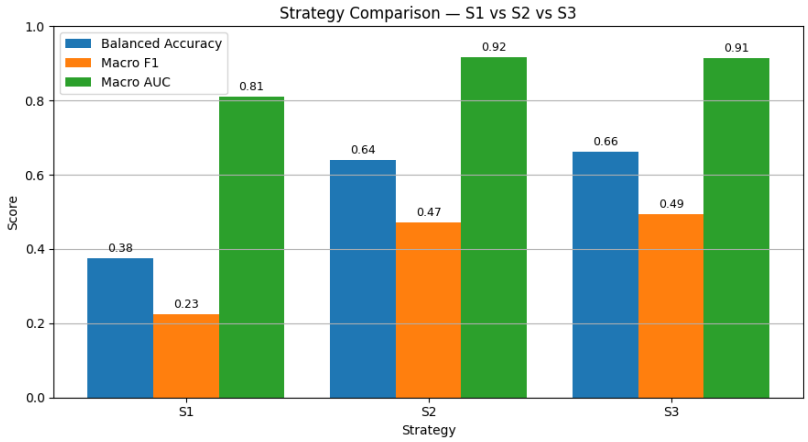


Figure 1: Cross-strategy comparison on the HAM10000 test split (EfficientNet-B0). The S1→S2 jump dominates; S3 adds a small further gain on balanced accuracy and macro F_1.

Why the model performs as it does. S1 collapses on the rare classes (per-class F_1 of 0.00 for both BCC and DF), confirming that ∼8k training images are insufficient to train ∼5M parameters from scratch Tajbakhsh et al. [2016]. S2 lifts balanced accuracy by $+26.4$ points by simply re-using ImageNet edges/textures and learning a 7-way linear head on top of them — the largest single jump in the table. S3 adds a further $+2.2$ points on balanced accuracy and $+2.3$ on macro F_1 by allowing the higher feature blocks to specialise toward dermoscopic texture while a reduced learning rate (10^{-4}) protects the lower blocks Yosinski et al. [2014].

Comparison to baseline / cross-architecture validation. Treating S1 as the from-scratch baseline, S2 and S3 both more than double its macro F_1. The S3-vs-S2 comparison, however, is *architecture-dependent*: on EfficientNet-B0, best validation accuracy was *higher* for S2 (67.3%) than for S3 (64.9%), even though S3 narrowly wins on the class-imbalance-aware test metrics in Table 2; on ResNet18, S3 (73.0% best val acc) clearly outperformed S2 (65.8%). We attribute the EfficientNet inversion to the very short 5-epoch budget — S3 only finishes unfreezing toward the end of training and so has little time to converge — and the ResNet result to the fact that ResNet18 benefits more from deeper adaptation to dermoscopic texture than the already-compact EfficientNet-B0 features do.

Failure cases. The dominant residual error mode for both S2 and S3 is the rare-class→NV confusion driven by the class prior (∼2/3 of training samples are NV). Class-weighted loss substantially

mitigates but does not eliminate this; per-class F_1 for DF remains the lowest at 0.18 (S2) / 0.31 (S3), reflecting only a few hundred DF training images.

4 Conclusion

What was achieved. A clean, reproducible comparison of three transfer-learning regimes on HAM10000 with two ImageNet backbones (EfficientNet-B0, ResNet18), with class-imbalance-aware training and evaluation throughout.

Key findings. (1) Re-using pre-trained features is essential — S1→S2 alone closes most of the gap. (2) Gradual unfreezing did *not* consistently outperform full freezing across architectures. The best validation accuracies were:

- EfficientNet-B0 – S3 (Gradual Unfreeze): 64.9%
- ResNet18 – S2 (Full Freeze): 65.8%
- ResNet18 – S3 (Gradual Unfreeze): **73.0%** (best overall).

On EfficientNet-B0, S2 reached a slightly higher validation accuracy than S3 (67.3% vs. 64.9%), most likely because the 5-epoch budget leaves S3 with too few epochs after unfreezing to converge. On ResNet18 the gap reverses and S3 clearly wins. (3) Putting (1) and (2) together: gradual unfreezing appears more beneficial for architectures that need stronger adaptation to the target domain (ResNet18 here), while a strong frozen backbone with a trained head can be sufficient — and even preferable under tight compute budgets — for architectures whose ImageNet features already transfer well (EfficientNet-B0).

Limitations. Only 5 epochs per run, no learning-rate schedule, no hyper-parameter sweep, single random seed, and no test-time augmentation/ensembling. The macro AUC of S3 is marginally below S2, hinting that gradual unfreezing reorganises the probability mass without strictly improving ranking quality.

Possible improvements. Longer training with cosine annealing; per-block discriminative learning rates rather than a single 10^{-4} post-unfreeze rate; oversampling rare classes (DF, VASC) in addition to weighted loss; and a stronger augmentation pipeline (colour jitter, dermoscopic-specific transforms) to attack the rare-class confusion identified in the failure analysis. Parameter-efficient adapters Hu et al. [2021] and lottery-ticket-style sparse fine-tuning Frankle and Carbin [2019], Morcos et al. [2019] are natural next steps for the partial-re-use regime.

References

Andre Esteva, Brett Kuprel, Roberto A Novoa, Justin Ko, Susan M Swetter, Helen M Blau, and Sebastian Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639):115–118, 2017.

Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Ari S Morcos, Haonan Yu, Michela Paganini, and Yuandong Tian. One ticket to win them all: generalizing lottery ticket initializations across datasets and optimizers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Nima Tajbakhsh, Jae Y Shin, Suryakanth R Gurudu, R Todd Hurst, Christopher B Kendall, Michael B Gotway, and Jianming Liang. Convolutional neural networks for medical image analysis: Full training or fine tuning? *IEEE Transactions on Medical Imaging*, 35(5):1299–1312, 2016.

Philipp Tschandl, Cliff Rosendahl, and Harald Kittler. The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific Data*, 5:180161, 2018.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

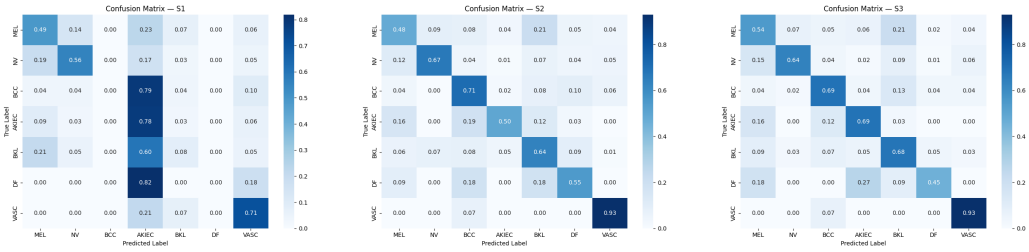
A Appendix

A.1 Per-class F_1 on the test split (EfficientNet-B0)

Strategy	MEL	NV	BCC	AKIEC	BKL	DF	VASC
S1	0.339	0.700	0.000	0.159	0.118	0.000	0.260
S2	0.415	0.790	0.510	0.492	0.543	0.176	0.377
S3	0.414	0.770	0.541	0.530	0.544	0.313	0.351

A.2 Confusion matrices (EfficientNet-B0)

The diagonal of the S1 confusion matrix is dominated by NV (the prior class) with whole rows for BCC, DF and VASC routed away from their true class. S2 already balances the diagonal substantially, and S3 further reduces the rare-class→NV leakage discussed in §3.

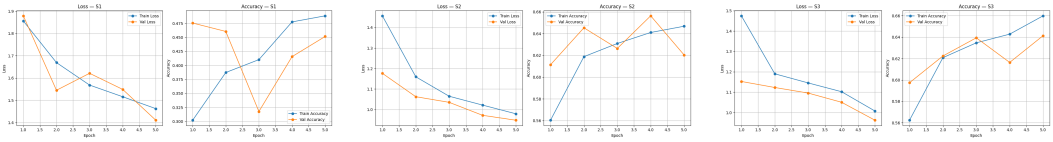


*S1 – From Scratch *S2 – Full Freeze *S3 – Gradual Unfreeze

Figure 2: Row-normalised confusion matrices on the HAM10000 test split (EfficientNet-B0).

A.3 Training curves (EfficientNet-B0)

The S3 curve shows the characteristic accuracy bumps after each unfreezing event, while S1 lags far below both pre-trained variants throughout training.



*S1 *S2 *S3

Figure 3: Per-strategy train/val loss and accuracy curves over the 5-epoch budget (EfficientNet-B0).

A.4 Reproducibility

All experiments are launched from the project root with the `skin-lesion` conda environment. Splits are exported once via `python data/export_splits.py`; the three strategies are then reproduced with `python train_s1.py`, `train_s2.py`, `train_s3.py`, or end-to-end with `python main.py`. Each run writes `results/{S1,S2,S3}_metrics.json`, the corresponding best checkpoint, training-curve PNG, and a normalised confusion matrix to `results/`.